

【实验报告】

课程名： 计算机组成与体系结构（H）

实验名称： lab4 CSR

学生： 杨箫羽

学号： 24300240069

时间： 2026 年 5 月 9 日

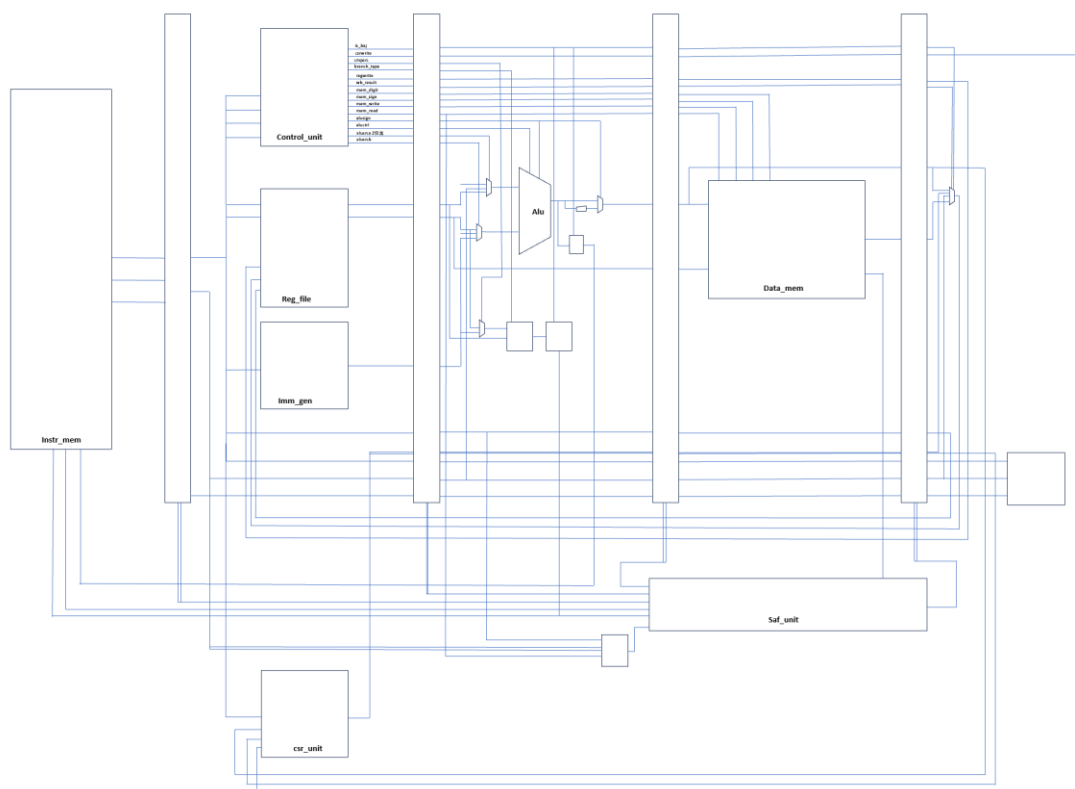
一、实验内容与实现思路说明

实验 3 要求增加 CSR 相关的指令。为了实现这些指令，我在 ID 阶段增加了 csr_file 的模块来实现 csr 寄存器值的存储，在 control_unit 中增加了 csrwrite 控制信号，同时为了解决数据冒险。在 verilator 上通过测试之后。

下面分总体电路图、新增的模块和做出重要修改的模块讲解、调试、bonus 问题回答四部分进行具体说明。

```
the image as /usr/local/bin/verilator --x86_64 --cc-toolchain
Using simulated 256MB RAM
Using /root/.26-Arch/ready-to-run/riscv64-nemu-interpretor-so for diffptest
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'clint' at [0x38000000, 0x3800ffff]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite' at [0x40600000, 0x4060000c]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite1' at [0x23333000, 0x2333300f]
The first instruction of core 0 has committed. Diffptest enabled.
[WARNING] diffptest store queue overflow
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x000000008001fff8
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 6196 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 32766
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 5288250 instr/s
Program execution has ended. To restart the program, exit NEMU and run again.
sh: 1: spike-dasm: not found
```

二、总体电路图



三、修改和新增的重要模块

1、csr_file

```
module csr_file import common::*;import csr_pkg::*;(
    input logic clk,
    input logic reset,
    input logic [31:0]instr_d,
    input logic [31:0]instr_w,
    input logic [11:0]new_csr_num,
    input logic [63:0] new_csr_value,
    input logic csrwrite,
    output logic [63:0] csr_value,
    output logic [11:0] csr_num,

    output logic [63:0] csr_mstatus,
    output logic [63:0] csr_mtvec,
    output logic [63:0] csr_mip,
    output logic [63:0] csr_mie,
    output logic [63:0] csr_mscratch,
    output logic [63:0] csr_mcause,
    output logic [63:0] csr_mtval,
    output logic [63:0] csr_mepc,
    output logic [63:0] csr_mcycle,
    output logic [63:0] csr_mhartid,
    output logic [63:0] csr_satp
);
```

//模块功能: 读取instr,将[31:20]位写入csr_num, 将这个csr号对应的寄存器的值写入csr_value
//当csrwrite为1时, 考虑instr[14:12]。
// 如果为001/101, 将new_csr_value写入new_csr_num对应的寄存器。
// 如果为010/110, 将new_csr_value中为1的位在new_csr_num对应的寄存器中置为1。
// 如果为011/111, 将new_csr_value中为1的位在new_csr_num对应的寄存器中置为0。

2、control unit

新增指令的控制信号和新增控制信号的含义如下表:

		csrww	csrrs	csrrc	csrrwi	csrrsi	csrrci
opcode		1110011	1110011	1110011	1110011	1110011	1110011
funct3		001	010	011	101	110	111
funct7							
bit30							
alusign	1时针对rv64i这种指令, 做32->64的扩展	0	0	0	0	0	0
aluctrl	(0-9) +/异或/或/与/-/有符号小于/无符号小于	0	0	0	0	0	0
alusrca	0为0, 1为rs1val,2为pc	1	1	1	0	0	0
alusrcb	0为rs2val, 1为imm, 3为0	0	0	0	1	1	1
regwrite	0不写, 1写	1	1	1	1	1	1
mem_write	0不写, 1写	0	0	0	0	0	0
mem_read	0不读, 1读	0	0	0	0	0	0
mem_sign	0时符号扩展, 1时零扩展	0	0	0	0	0	0
mem_digit	8/16/32/64	0	0	0	0	0	0
wb_result	0为alu结果, 1为mem读结果, 2为pc+4, 3为pc+4+imm	3	3	3	3	3	3
branch_type	0-5分别代表=, !=, 有符号<, 有符号>=, 无符号<, 无符号>=	0	0	0	0	0	0
cmpsrc	0为rs2val, 1为imm	0	0	0	0	0	0
is_baj	0代表都不是, 1代表b类, 2代表jal,3代表jalr	0	0	0	0	0	0
csrwrite	0不写, 1写	1	1	1	1	1	1

四、调试

1、一开始一直停在第一条指令的 csr 写回。后发现有一个问题是, csrfile 里面两次使用了 instr, 一次是读取新的 csrnum, 一次是根据 csr 的 funct3 写回。两个 instr 并不是一个阶段。一开始没想到, 用了一个 instr, 所以一直是错的。

2、我一开始的设计思路是用 alu 来分别算 $imm+0$ 和 $0+rs1val$ ，但是因为译码的位置不对所以 aluresult 一直不对。后来发现可以直接在 datapath 里面实现 csr 的 operand 的获取，所以没有在 alu 里面实现，直接在 datapath 里完成了。

五、bonus 问题

1、各个 csr 寄存器作用

Mstatus: 机器模式的状态寄存器，保存全局的中断使能、特权级返回状态等状态。

Mtvec: 机器模式 trap 的入口地址，跳转到这里

Mip: 机器模式表示中断挂起

Mie: 机器模式控制那些中断可以响应

Mscratch: 机器模式临时寄存器，常用于保存通用寄存器/栈指针

Mcause: 记录进入机器模式的 trap 的原因

Mtval: trap 附加的信息，比如出错地址

Mepc: 记录 trap 前的 pc

Mcycle: 机器模式周期计数器

Mhartid: 当前 core 编号

Satp: 特权地址翻译与保护寄存器，保存页表根地址、分页模式等。

2、为什么 csr 写后要刷新寄存器？

因为 csr 是全局控制状态，而不是只影响某个数据操作数的普通寄存器。如果不 flush，后面的指令会在旧的 csr 的状态下完成（如取指、译码、权限判断、终端判断、异常处理入口选择等），造成不一致。